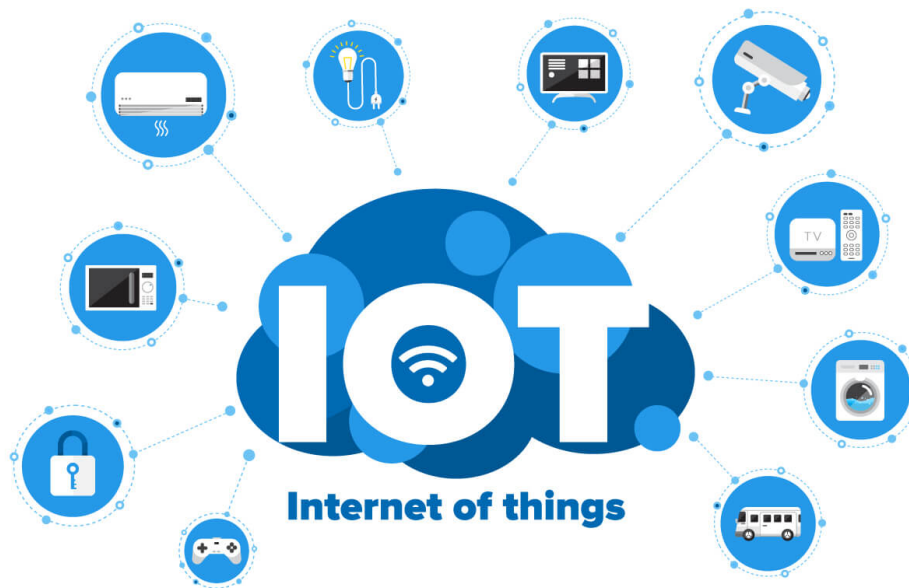


Motor Protection System

Second Project



*Dana Natsheh
Cyber Robot
IoT*

1. Executive Summary

This report documents the design, implementation, and testing of a Simple Motor Protection System built around the Espressif ESP32 microcontroller. The system energizes a DC motor through a relay, continuously monitors the motor's current draw using a Hall-effect current sensor (ACS712-family, 30 A range), and autonomously disconnects the motor whenever the current exceeds a configurable overcurrent threshold. A physical push-button provides an immediate, software-debounced Emergency Stop (E-Stop) that can interrupt the motor at any time, and four status LEDs (Running, Stopped, High Current, Low Current) give an at-a-glance view of system state.

The firmware is implemented as a single-file Arduino/PlatformIO sketch (`main.cpp`) organized around a finite-state machine with four states: `STOPPED`, `RUNNING`, `LOW_CURRENT`, and `FAULT_HIGH`. A self-calibration routine measures the sensor's true zero-current baseline at boot (with the motor guaranteed off) rather than assuming a fixed textbook value – this turned out to be the single most important design decision in the project, and is discussed in detail in Section .

2. Hardware Architecture & Pin Mapping

The project uses an Espressif ESP32 DevKit v1 board powered over USB. All digital and analog peripherals are referenced to the board's onboard 3.3 V logic rail. The motor itself is powered from a separate bench power supply, switched by a relay module – the motor's power path is *not* referenced to the ESP32's 3.3 V.

Resistor sizing note

All four LEDs must be wired with an inline current-limiting resistor (not present in earlier breadboard photos from Project 1 – corrected here). For a standard 3.3 V GPIO output driving a common 2 V red/yellow/green LED at ~ 10 mA:

$$R = \frac{V_{GPIO} - V_{LED}}{I_{LED}} = \frac{3.3\text{V} - 2.0\text{V}}{0.01\text{A}} \approx 130\ \Omega$$

A standard $220\ \Omega$ resistor is used on all four channels for a safe margin and to keep brightness comfortable to view continuously.

3. Schematic and Circuit

Peripheral	Peripheral Pin	ESP32 GPIO	Electrical Logic / Notes
Relay Module	IN	GPIO 23	Digital Output, active-LOW (RELAY_ACTIVE_LOW = true)
Relay Module	VCC / GND	5V / GND	Powered from board 5V rail (check module's coil voltage)
Current Sensor (ACS712)	VOUT	GPIO 34	Analog Input, 12-bit ADC, 0–3.3 V
Current Sensor (ACS712)	VCC	3.3V	See Section – ratiometric sensor caveat
Current Sensor (ACS712)	GND	GND	Common ground reference
E-Stop Push Button	Signal	GPIO 21	Digital Input, INPUT_PULLUP, active-LOW, 40 ms debounce
LED – Running (Green)	Anode (+)	GPIO 12	Digital Output through current-limiting resistor
LED – Stopped (Yellow)	Anode (+)	GPIO 19	Digital Output through current-limiting resistor
LED – High Current (Red)	Anode (+)	GPIO 33	Digital Output through current-limiting resistor
LED – Low Current (Blue)	Anode (+)	GPIO 27	Digital Output through current-limiting resistor
All LED Cathodes	(–)	GND	Common ground reference

Table 1: ESP32 pin mapping for the motor protection system.

Schematic

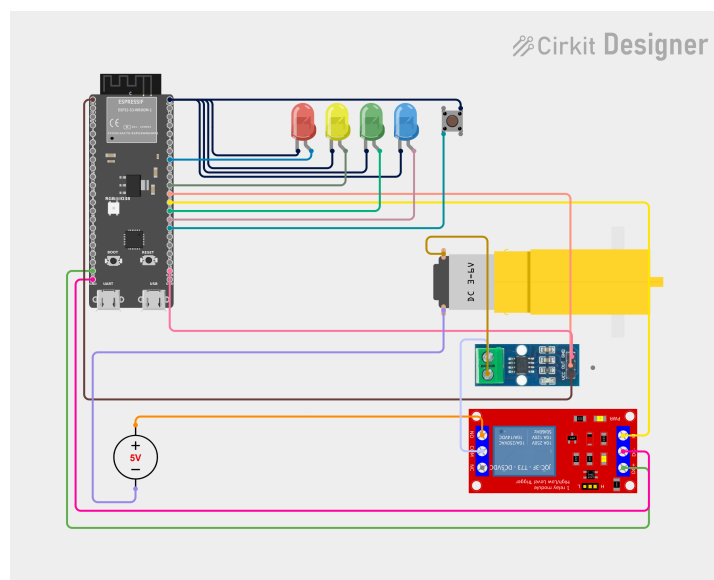


Figure 1: Full wiring schematic.

Assembled Circuit

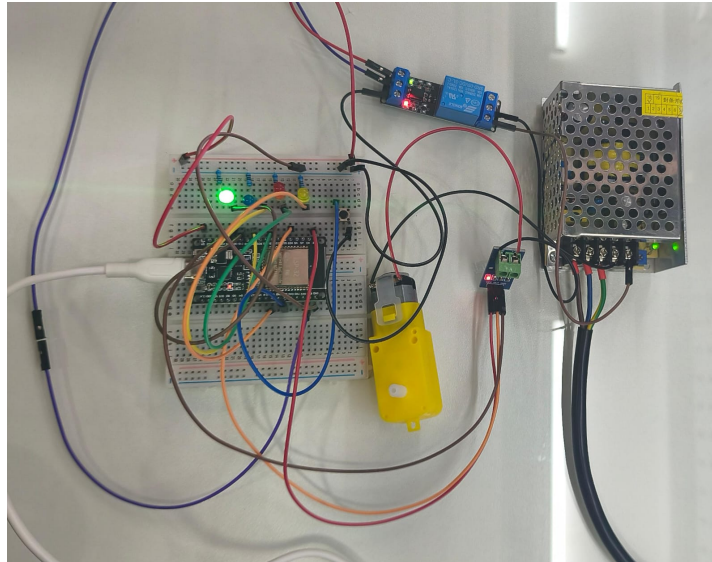
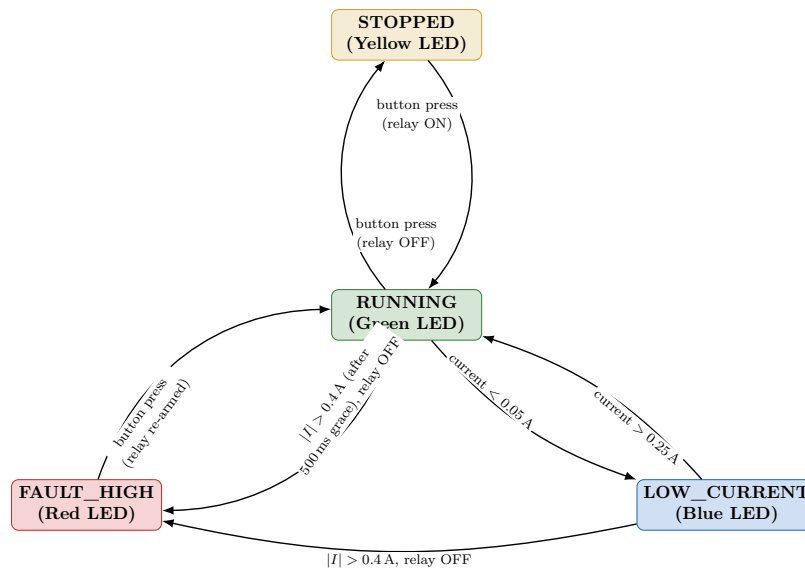


Figure 2: Assembled breadboard circuit under test.

4. System State Machine

The firmware is organized as an explicit finite-state machine (`enum SystemState`), which is what keeps the control logic predictable and easy to extend.



5. Tested Circuit & Serial Monitor Output

Sample Output

```
* Executing task in folder project_2: C:\Users\User\.platformio\penv\Scripts\platformio.exe device
monitor --port COM3
Current: 0.312 A
Current: 0.301 A
Current: 0.291 A
Current: 0.326 A
E-stop pressed. Motor stopped.
Current: -0.005 A
Current: 0.055 A
Current: -0.024 A
Current: -0.032 A
Current: -0.018 A
Current: -0.025 A
Current: 0.041 A
Current: 0.013 A
Restart requested. Re-enabling motor and re-checking current.
Current: 1.168 A
Current: 0.508 A
Current: 0.320 A
Current: 0.288 A
Current: 0.247 A
Current: 0.248 A
```

Figure 3: PlatformIO Serial Monitor output at 115200 baud.

Expected console behaviour

- On boot: self-test flash of all 4 LEDs (2s), then the auto-calibration banner and measured Zero voltage = X.XXXX V.
- While STOPPED: only the Yellow LED is lit; current is read every 50 ms but ignored by the state machine.
- After pressing the button (motor starts): Green LED on, relay energized, first 500 ms of current readings are ignored (motor inrush/startup surge).
- Steady low load: state drops to LOW_CURRENT, Blue LED on.
- Induced overcurrent (e.g. stall the motor by hand): Red LED on, relay opens within one 50 ms sample interval, motor stops instantly.
- Button press while in FAULT_HIGH: clears the fault and restarts the motor.

LED	Meaning	Pin
Green (Running)	Motor energized, current within normal band	GPIO 12
Yellow (Stopped)	Relay open, motor fully off (idle or E-Stopped)	GPIO 19
Red (High Current)	Overcurrent trip latched, relay open, awaiting reset	GPIO 33
Blue (Low Current)	Motor energized but drawing less than the low threshold	GPIO 27

6. How the System Works

1. **Boot-time self-calibration.** Before anything else, the relay is forced OFF (fail-safe default) and held off for 1 s. The firmware then samples the current sensor's raw ADC output 300 times (2 ms apart) with the motor guaranteed to be drawing zero current, and stores the averaged voltage as `ZERO_CURRENT_VOLTAGE`. Every later current calculation is referenced to this measured value instead of a hardcoded constant.
2. **Debounced E-Stop / Start button.** `updateButton()` samples the button every loop iteration and only accepts a state change once the reading has been stable for 40 ms, filtering mechanical contact bounce. A rising-to-falling transition sets `buttonPressEvent`, consumed once per press by `handleButtonPress()`.
3. **Continuous current sampling.** Every 50 ms, `readCurrentAmps()` averages 50 raw ADC samples, converts to volts, subtracts the calibrated zero point, and divides by the sensor's sensitivity (`SENSITIVITY_V_PER_A`) to get amps.
4. **Startup-surge grace period.** For 500 ms after the motor is (re)started, an overcurrent condition is ignored, since brushed DC motors draw a large inrush current at the moment of energizing that is not representative of steady-state load.
5. **Overcurrent trip.** Outside the grace period, if $|I| > \text{HIGH_CURRENT_THRESHOLD_A}$ (0.4 A), the relay is opened immediately and the state latches to `FAULT_HIGH` until the button is pressed again.
6. **Low-current detection with hysteresis.** While running, the state toggles between `RUNNING` and `LOW_CURRENT` using a hysteresis band around `LOW_CURRENT_THRESHOLD_A` (0.15 A $\pm$ 0.1 A) so that noise near the threshold does not cause LED flicker.
7. **LED reflection of state.** `updateLEDs()` is called once per `loop()` and simply lights the single LED that corresponds to `currentState` – no LED logic is duplicated elsewhere in the code.

7. Software Implementation

The complete, current firmware is listed below for reference and reproducibility. It is a single translation unit (`main.cpp`) targeting the Arduino core for ESP32 under PlatformIO.

```

1 #include <Arduino.h>
2 #include <math.h>
3
4 // ----- PIN ASSIGNMENTS -----
5 const int PIN_RELAY          = 23;
6 const int PIN_CURRENT_SENSOR = 34;
7 const int PIN_BUTTON         = 21;
8
9 const int PIN_LED_RUNNING    = 12; // Green
10 const int PIN_LED_STOPPED    = 19; // Yellow

```

```

11 const int PIN_LED_HIGH_CURRENT = 33; // Red
12 const int PIN_LED_LOW_CURRENT  = 27; // Blue
13
14 unsigned long motorStartTime = 0;
15
16 // ----- CURRENT SENSOR CALIBRATION -----
17 const float ADC_MAX_VOLTAGE      = 3.3f;
18 const int   ADC_RESOLUTION       = 4095;
19 float ZERO_CURRENT_VOLTAGE = 0.0f;
20 const float SENSITIVITY_V_PER_A  = 0.066f;
21
22 const int CURRENT_SAMPLE_COUNT = 50;
23
24 // ----- THRESHOLDS -----
25 const float HIGH_CURRENT_THRESHOLD_A = 0.4f;
26 const float LOW_CURRENT_THRESHOLD_A  = 0.15f;
27 const float CURRENT_HYSTERESIS_A    = 0.1f;
28
29 // ----- RELAY POLARITY -----
30 const bool RELAY_ACTIVE_LOW = true;
31
32 // ----- TIMING -----
33 const unsigned long BUTTON_DEBOUNCE_MS = 40;
34 const unsigned long SAMPLE_INTERVAL_MS = 50;
35
36 // ----- SYSTEM STATES -----
37 enum SystemState {
38     STATE_STOPPED,
39     STATE_RUNNING,
40     STATE_LOW_CURRENT,
41     STATE_FAULT_HIGH
42 };
43
44 SystemState currentState = STATE_STOPPED;
45
46 // ----- BUTTON DEBOUNCE STATE -----
47 bool lastButtonReading  = HIGH;
48 bool debouncedButtonState = HIGH;
49 unsigned long lastButtonChangeMs = 0;
50 bool buttonPressEvent = false;
51
52 unsigned long lastSampleMs = 0;
53
54 // ----- FORWARD DECLARATIONS -----
55 void updateButton();
56 void handleButtonPress();
57 float readCurrentAmps();
58 void evaluateCurrent(float currentA);
59 void setRelay(bool energized);
60 void updateLEDs();
61 void calibrateCurrentSensor();
62
63 // ----- AUTO CALIBRATION -----
64 void calibrateCurrentSensor() {
65     const int CALIBRATION_SAMPLES = 300;
66     uint32_t adcSum = 0;
67
68     Serial.println();
69     Serial.println("=====");
70     Serial.println("Current sensor auto calibration");

```

```
71 Serial.println("Motor must be OFF...");
72 Serial.println("=====");
73
74 delay(1000);
75
76 for (int i = 0; i < CALIBRATION_SAMPLES; i++) {
77     adcSum += analogRead(PIN_CURRENT_SENSOR);
78     delay(2);
79 }
80
81 float adcAverage = adcSum / (float)CALIBRATION_SAMPLES;
82 ZERO_CURRENT_VOLTAGE = (adcAverage / ADC_RESOLUTION) * ADC_MAX_VOLTAGE;
83
84 Serial.print("Zero voltage = ");
85 Serial.print(ZERO_CURRENT_VOLTAGE, 4);
86 Serial.println(" V");
87 Serial.println("Calibration complete.");
88 Serial.println();
89 }
90
91 void setup() {
92     Serial.begin(115200);
93
94     pinMode(PIN_RELAY, OUTPUT);
95     pinMode(PIN_BUTTON, INPUT_PULLUP);
96
97     pinMode(PIN_LED_RUNNING, OUTPUT);
98     pinMode(PIN_LED_STOPPED, OUTPUT);
99     pinMode(PIN_LED_HIGH_CURRENT, OUTPUT);
100    pinMode(PIN_LED_LOW_CURRENT, OUTPUT);
101
102    analogReadResolution(12);
103
104    // Quick boot self-test flash: all 4 LEDs on for 2s, then off.
105    digitalWrite(PIN_LED_RUNNING, HIGH);
106    digitalWrite(PIN_LED_STOPPED, HIGH);
107    digitalWrite(PIN_LED_HIGH_CURRENT, HIGH);
108    digitalWrite(PIN_LED_LOW_CURRENT, HIGH);
109    delay(2000);
110    digitalWrite(PIN_LED_RUNNING, LOW);
111    digitalWrite(PIN_LED_STOPPED, LOW);
112    digitalWrite(PIN_LED_HIGH_CURRENT, LOW);
113    digitalWrite(PIN_LED_LOW_CURRENT, LOW);
114
115    // Motor OFF first (fail-safe), then calibrate current sensor while it's guaranteed at 0
116    A
117    setRelay(false);
118    delay(1000);
119
120    calibrateCurrentSensor();
121
122    currentState = STATE_STOPPED;
123    updateLEDs();
124
125    Serial.println("Motor protection system booted. State: STOPPED.");
126 }
127
128 void loop() {
129     updateButton();
```

```

130  if (buttonPressEvent) {
131      handleButtonPress();
132      buttonPressEvent = false;
133  }
134
135  if (millis() - lastSampleMs >= SAMPLE_INTERVAL_MS) {
136      lastSampleMs = millis();
137      float currentA = readCurrentAmps();
138      Serial.print("Current: ");
139      Serial.print(currentA, 3);
140      Serial.println(" A");
141      evaluateCurrent(currentA);
142  }
143
144  updateLEDs();
145  }
146
147  // ----- BUTTON HANDLING -----
148  void updateButton() {
149      bool reading = digitalRead(PIN_BUTTON);
150
151      if (reading != lastButtonReading) {
152          lastButtonChangeMs = millis();
153      }
154
155      if ((millis() - lastButtonChangeMs) > BUTTON_DEBOUNCE_MS) {
156          if (reading != debouncedButtonState) {
157              debouncedButtonState = reading;
158              if (debouncedButtonState == LOW) {
159                  buttonPressEvent = true;
160              }
161          }
162      }
163
164      lastButtonReading = reading;
165  }
166
167  void handleButtonPress() {
168      switch (currentState) {
169          case STATE_RUNNING:
170          case STATE_LOW_CURRENT:
171              setRelay(false);
172              currentState = STATE_STOPPED;
173              Serial.println("E-stop pressed. Motor stopped.");
174              break;
175
176          case STATE_STOPPED:
177          case STATE_FAULT_HIGH:
178              setRelay(true);
179              motorStartTime = millis();
180              currentState = STATE_RUNNING;
181              Serial.println("Restart requested. Re-enabling motor and re-checking current.");
182              break;
183      }
184  }
185
186  // ----- CURRENT SENSING -----
187  float readCurrentAmps() {
188      long rawSum = 0;
189      for (int i = 0; i < CURRENT_SAMPLE_COUNT; i++) {

```

```

190     rawSum += analogRead(PIN_CURRENT_SENSOR);
191 }
192 float rawAvg = rawSum / (float)CURRENT_SAMPLE_COUNT;
193 float voltage = (rawAvg / ADC_RESOLUTION) * ADC_MAX_VOLTAGE;
194 float current = -(voltage - ZERO_CURRENT_VOLTAGE) / SENSITIVITY_V_PER_A;
195 return current;
196 }
197
198 void evaluateCurrent(float currentA) {
199     if (currentState == STATE_STOPPED) {
200         return;
201     }
202
203     if (currentState == STATE_FAULT_HIGH) {
204         return;
205     }
206
207     // Ignore startup surge for the first 500 ms after (re)starting
208     if (millis() - motorStartTime > 500 && fabs(currentA) > HIGH_CURRENT_THRESHOLD_A) {
209         setRelay(false);
210         currentState = STATE_FAULT_HIGH;
211         Serial.print("OVERCURRENT TRIP: ");
212         Serial.print(currentA);
213         Serial.println(" A. Relay opened. Press button to reset.");
214         return;
215     }
216
217     if (fabs(currentA) < (LOW_CURRENT_THRESHOLD_A - CURRENT_HYSTERESIS_A)) {
218         currentState = STATE_LOW_CURRENT;
219     } else if (fabs(currentA) > (LOW_CURRENT_THRESHOLD_A + CURRENT_HYSTERESIS_A)) {
220         currentState = STATE_RUNNING;
221     }
222 }
223 }
224
225 // ----- OUTPUTS -----
226 void setRelay(bool energized) {
227     bool pinState = RELAY_ACTIVE_LOW ? !energized : energized;
228     digitalWrite(PIN_RELAY, pinState ? HIGH : LOW);
229 }
230
231 void updateLEDs() {
232     digitalWrite(PIN_LED_RUNNING,      currentState == STATE_RUNNING      ? HIGH : LOW);
233     digitalWrite(PIN_LED_STOPPED,      currentState == STATE_STOPPED      ? HIGH : LOW);
234     digitalWrite(PIN_LED_HIGH_CURRENT, currentState == STATE_FAULT_HIGH  ? HIGH : LOW);
235     digitalWrite(PIN_LED_LOW_CURRENT,  currentState == STATE_LOW_CURRENT  ? HIGH : LOW);
236 }

```

Listing 1: main.cpp – full motor protection firmware

8. Bill of Materials

Item	Qty	Notes
ESP32 DevKit v1	1	Main controller
ACS712 current sensor module (30A variant)	1	Sensitivity 66 mV/A; see Section on 3.3V operation
5V single-channel relay module	1	Confirm active-LOW / active-HIGH before wiring RELAY_ACTIVE_LOW
Momentary push button	1	Wired to INPUT_PULLUP, other leg to GND
LEDs: green, yellow, red, blue	1 each	5mm standard LEDs
220 Ω resistor	4	One per LED
DC motor (brushed)	1	Load under test
Bench power supply	1	Sized to motor's rated voltage/current
Breadboard + jumper wires	–	–

9. What I Learned

The current sensor is ratiometric to its supply voltage – and I was powering it at 3.3 V instead of its rated 5 V

The ACS712 family (like most Hall-effect current sensor breakout boards) is a ratiometric device: both its zero-current output and its sensitivity are specified relative to its supply voltage, V_{CC} . The datasheet numbers most people quote – a $V_{CC}/2$ (i.e. 2.5 V) zero point and a fixed sensitivity such as 66 mV/A for the 30 A part – are only valid *when the sensor is actually powered from a 5 V rail*, which is its rated operating voltage.

In this project the sensor's V_{OUT} feeds directly into an ESP32 ADC pin, and ESP32 ADC pins are only rated for 0–3.3 V. Two options exist for bridging this gap:

1. Power the sensor from 5 V (as intended by the datasheet) and add a resistor-divider or level shifter on V_{OUT} before it reaches the ESP32 pin, or
2. Power the sensor directly from the ESP32's 3.3 V rail, which keeps the output naturally within ADC range but shifts *both* the zero point and the sensitivity, since the part is ratiometric.

This project uses option (2) for wiring simplicity, and that decision is the source of most of the debugging done on this task:

- **Zero point drifts with supply voltage and with each board.** A hardcoded “2.5 V is zero current” assumption produced current readings that were wrong by a large, constant offset (and different from board to board / breadboard to breadboard). The fix implemented here is `calibrateCurrentSensor()`: at every boot, with the relay forced open and the motor guaranteed to be drawing zero current, the firmware averages 300 ADC samples and stores that measured voltage as the actual zero point (`ZERO_CURRENT_VOLTAGE`). This makes the system self-correcting for supply drift, board-to-board variation, and running at 3.3 V instead of 5 V, without needing to hand-tune a constant.

- **Sensitivity also scales with V_{CC} , and this is the part that is easy to miss.** The 66 mV/A figure used in this code (`SENSITIVITY_V_PER_A`) is the 5 V datasheet value, applied as-is while the sensor is actually running at 3.3 V. Because sensitivity is ratiometric, the true sensitivity at 3.3 V is closer to $66\text{mV/A} \times (3.3/5) \approx 43.6\text{mV/A}$. Leaving the constant at the 5 V value means the calculated amps are systematically under-scaled relative to real current – the trip point still works as a *relative* threshold (it was tuned empirically against a known overload), but the printed “Current: X.XXX A” values on the serial monitor should not be trusted as absolute, calibrated amps until this constant is corrected or the sensor is moved to a proper 5 V + divider supply. **This is the top item for the next person to fix** (see Section ??).
- **Averaging matters more at 3.3 V.** Because the usable ADC swing around the zero point is smaller at 3.3 V than at 5 V, ADC quantization noise has proportionally more effect on the computed current. Averaging 50 samples per reading (`CURRENT_SAMPLE_COUNT`) and 300 samples for the boot calibration (`CALIBRATION_SAMPLES`) was necessary to get a stable enough baseline for the hysteresis logic to avoid chattering near the low-current threshold.
- **Startup inrush looks like a fault if you don’t grace it.** Early testing tripped the overcurrent protection on every motor start, because the initial inrush current of a stalled/starting brushed DC motor is several times its running current. The 500 ms grace window in `evaluateCurrent()` (measured from `motorStartTime`) fixed this without weakening the steady-state protection.

Other takeaways

- A fail-safe boot sequence (force relay OFF, *then* calibrate, *then* allow RUNNING) prevents the motor from ever being energized with an uncalibrated or undefined current baseline.
- Debouncing in software (40 ms stability window) was simpler and more reliable than adding an RC debounce circuit, and kept the button wiring to two wires.
- Keeping LED output logic in a single function (`updateLEDs()`) driven purely by the `currentState` enum avoided the bug class where two LEDs are lit at once because their conditions were updated in different places.